

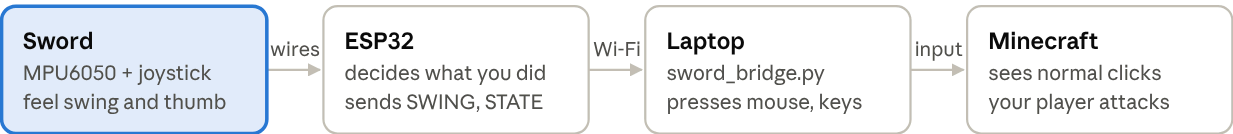
Motion Sword Build Guide

Oct 2, 2026 · @Jevan

How it works

The sword never talks to Minecraft directly. It pretends to be your mouse and keyboard, so you don't have to change Minecraft at all.

The sword pretends to be your mouse and keyboard



how a swing becomes an attack · 4 stages

1. **The motion sensor (MPU6050)** feels how fast the sword moves. A hard swing makes a big spike in acceleration.
2. **The joystick** on the handle reads which way your thumb pushes.
3. **The ESP32** is the tiny computer that reads both, decides what you did (swing, block, walk, jump), and sends a short text message like `SWING 7` over Wi-Fi.
4. **A Python script on the laptop** (sword_bridge.py) catches those messages and presses the real mouse button and keys.
5. **Minecraft** sees a left click and your character attacks.

You do this with the sword	The laptop presses	In Minecraft
Swing hard	Left click	Attack
Hold it sideways and still	Hold right click	Shield (in offhand) blocks
Push the joystick	Hold W, A, S or D	Walk
Press the joystick down	Hold Space	Jump

Why this design: pretending to be a mouse means zero Minecraft mods, it works on any version, and the only code you write is in two small files.

What you need

You already have almost everything. The only thing to grab is a 9V battery clip with bare wires.

Part	Where it comes from	What it does
ESP32 dev board (IoT Crazy, USB-C, CP2102)	You bought it	The brain. Reads sensors, sends Wi-Fi messages
MPU6050 (GY-521, pins need soldering)	You bought it	Feels the swing
Joystick module	ELEGOO kit	Walking and jumping
Breadboard	ELEGOO kit	Connects wires without soldering
Male-to-male jumper wires	ELEGOO kit	Wires between breadboard holes
Female-to-male DuPont wires (10)	ELEGOO kit	Plug onto the sensor and joystick pins so they can sit away from the breadboard
USB-C data cable	Your own	Uploading code and testing
9V battery + clip with bare wires (or a USB power bank)	Your own	Powers the sword with no cable to the laptop
Phone with hotspot	Your own	Private Wi-Fi for the sword and laptop
Laptop with Minecraft Java	Your own	Runs the game and the Python script
Foam board or cardboard, paint, tape, zip ties	Craft store	The sword itself

From the kit you use the breadboard, the wires and the joystick. You can leave the UNO board itself in the box, since the ESP32 does its job. **Your 9V battery is fine**, as long as it only connects to the ESP32's **VIN** pin (Step 7 shows how). Never put 9V on the 3V3 pin or

the + rail.

Code files (download from the chat):

- `sword_test.ino` : tests the motion sensor only
- `sword_wireless.ino` : the full sword code that goes on the ESP32
- `sword_bridge.py` : the laptop script that clicks for you

Step 1: Set up the laptop

Do this once, on the laptop that will run Minecraft. It takes about 20 minutes.

1. **Install the Arduino IDE** from arduino.cc.

- *Why*: this is the app that sends your code into the ESP32.

2. **Add ESP32 support.** Open File → Preferences. In *Additional boards manager URLs* paste `https://espressif.github.io/arduino-esp32/package_esp32_index.json`. Then open Tools → Board → Boards Manager, search **esp32**, and install the one by **Espressif Systems**.

- *Why*: the Arduino IDE only knows UNO-style boards out of the box. This teaches it how to talk to an ESP32.

3. **Install the sensor library.** Tools → Manage Libraries, search **Adafruit MPU6050**, install it, and click *Install all* when it asks about extra libraries.

- *Why*: a library is code someone already wrote for reading this exact sensor, so you don't have to.

4. **Plug in the ESP32** with the USB-C cable. Under Tools → Port a new port should appear (COM3 or similar on Windows, `/dev/cu.usbserial...` on Mac).

- No new port? Your board uses a **CP2102** chip. Install the *CP210x USB to UART driver* from Silicon Labs, restart, and try a different cable. Many cables only charge.

5. **Pick the board.** Tools → Board → esp32 → **ESP32 Dev Module**.

6. **Install Python 3** from python.org. On Windows, tick *Add Python to PATH* in the installer.

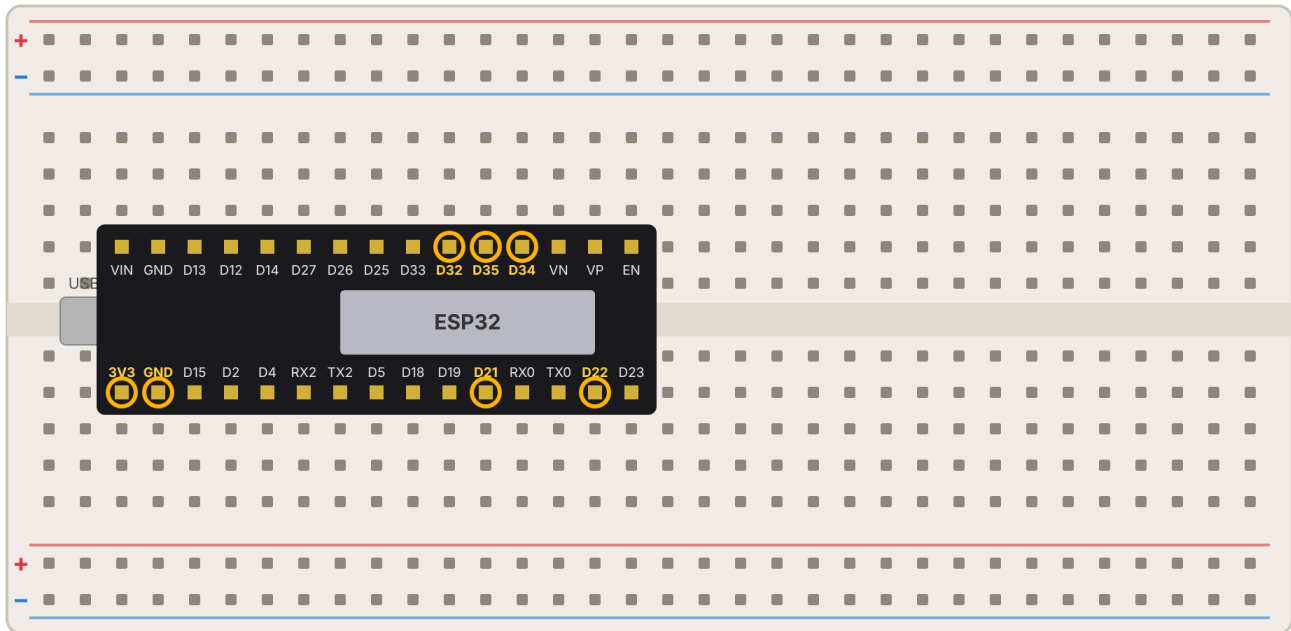
7. **Install the two Python libraries.** Open a terminal (Command Prompt on Windows, Terminal on Mac) and run `pip install pyserial pynput`.

- *Why*: `pyserial` lets Python listen to the USB cable, and `pynput` lets it press the mouse and keyboard.

Done when: the ESP32 shows up under Tools → Port.

Step 2: Place the ESP32 on the breadboard

The ESP32 sits across the groove in the middle, with the USB port sticking off one end.



How a breadboard works (the one idea you need):

- Each short column of 5 holes on one side of the groove is joined by a hidden metal strip. Two wires in the same column of 5 are connected.
- The groove in the middle splits each column. The top 5 and bottom 5 are **not** connected.
- The long rows along the edges (marked + red and – blue) are joined all the way across. These are the **power rails**.

Do this:

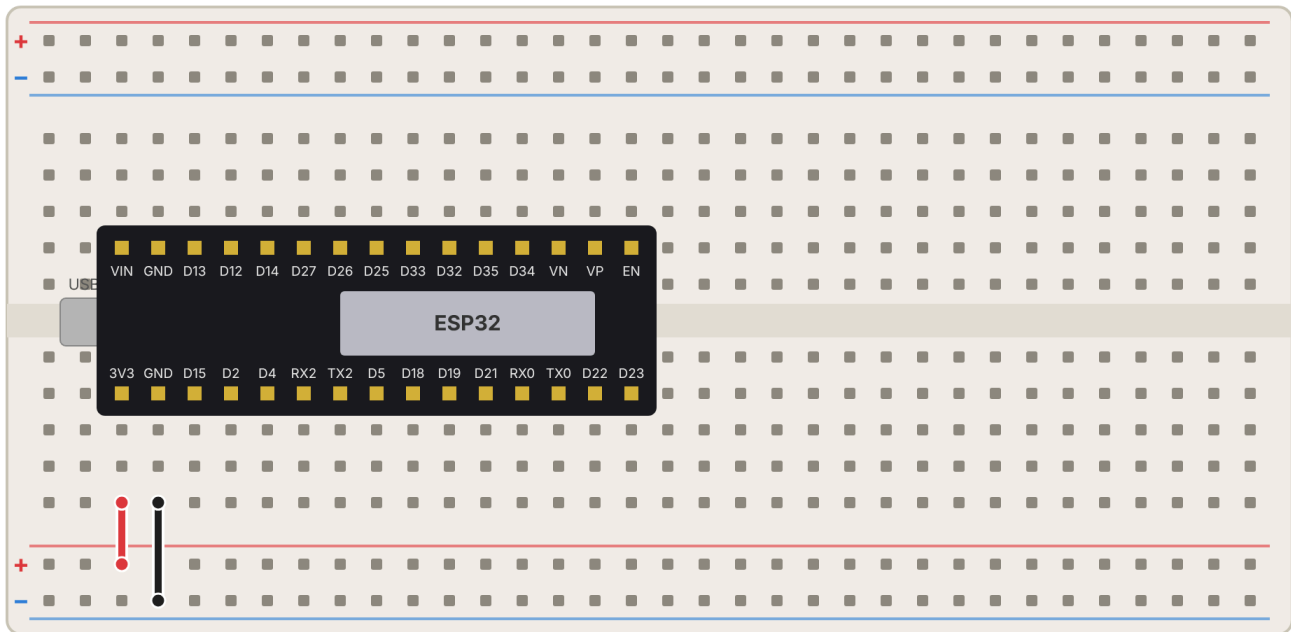
1. Press the ESP32 gently into the board so its top pins are on one side of the groove and its bottom pins on the other. Your team already did this.
2. Read the tiny labels next to the pins. You will use **3V3**, **GND**, **D21**, **D22** (bottom row) and **D32**, **D35**, **D34** (top row).
3. Check that each of those pins has **at least one free hole** in its column. That free hole is where its wire goes.

Why across the groove: if both rows of pins sat on the same side, the metal strips would connect every top pin to the pin below it and short the board.

Problem: no free holes on one side? This ESP32 is wide, so on some breadboards it covers almost every hole. Fix it one of two ways: slide the ESP32 so one row of pins hangs off the edge of the breadboard and connect those pins with female-to-male wires, or put two breadboards side by side with the ESP32 across the gap between them.

Step 3: Connect the power rails

Two wires turn the bottom edge of the breadboard into a power strip that both the sensor and joystick can plug into.



1. **Red wire:** from a free hole in the **3V3** column to the bottom + rail.
2. **Black wire:** from a free hole in the **GND** column (the one next to 3V3) to the bottom - rail.

What these words mean:

- **3V3 (3.3 volts)** is the positive power, like the + end of a battery.
- **GND (ground)** is the negative side, like the - end. Every part needs both, or no electricity flows through it.

Why rails: the ESP32 has only one 3V3 pin, but two parts need power. Running it to a rail gives you a whole row of 3.3V holes to share.

Why 3.3V and never 5V: the ESP32's pins only handle 3.3V. If the sensor or joystick ran on 5V, they would send signals that are too strong for it. Leave the **VIN** pin alone.

Wire colours don't change anything electrically. Red for power and black for ground is the habit everyone uses so mistakes are easy to spot.

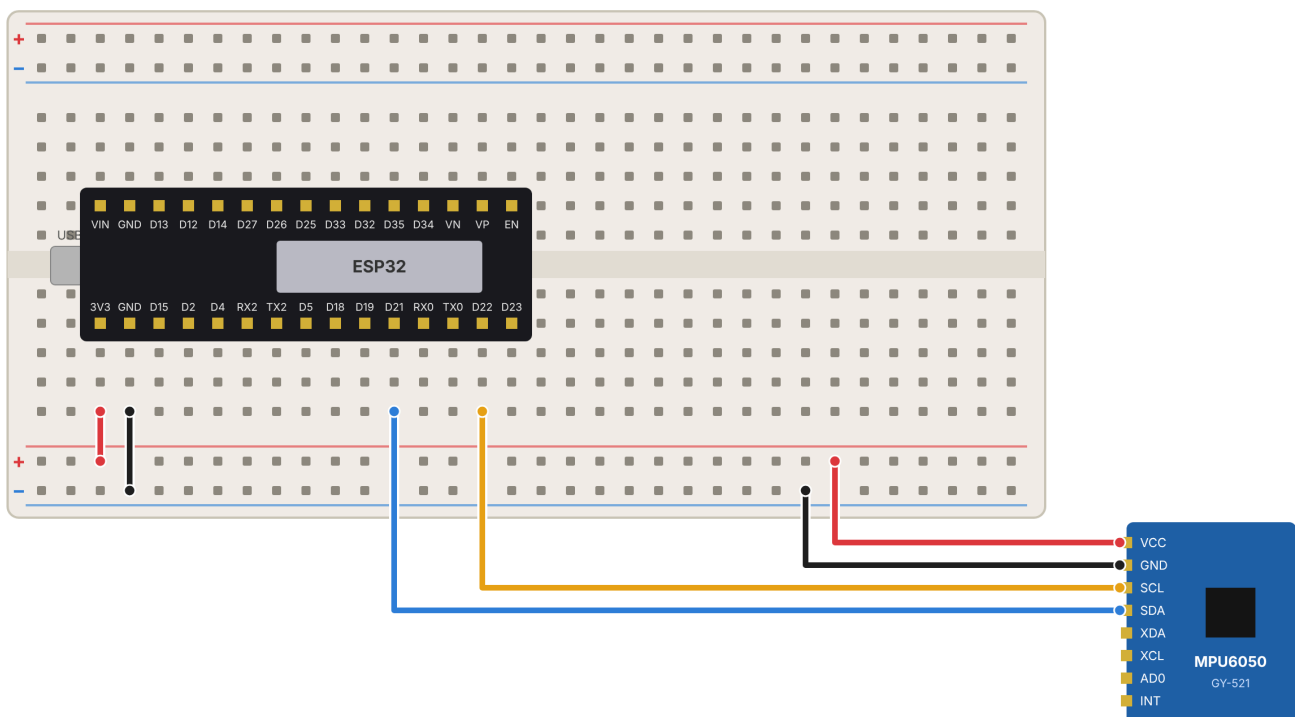
Step 4: Wire and test the motion sensor

First: solder the pins on. Your sensor came with loose pin strips (the black plastic strips). Pins that are only pushed in touch on and off, and every wobble looks like a swing. Ask the hackathon hardware table for a soldering iron and a helper.

1. Use the **straight** strip. Snap off a piece of 8 pins if it's longer.
2. Push the **long** ends of the strip into the breadboard to hold it steady.
3. Set the sensor on top, so the short ends poke through its 8 holes, labels facing up.
4. Touch the iron to each pin and its gold pad for 2 seconds, then feed in a little solder. Each joint should look like a small shiny cone.
5. Check that no blob of solder joins two pins together.

Safety: the iron tip is about 350°C. Hold it only by the handle, put it back in its stand every time, and keep the fumes out of your face.

Four female-to-male wires connect the MPU6050. Then you upload a test and watch the numbers jump when you swing.



Push the **female** end onto the sensor pin and the **male** end into the breadboard.

Sensor pin	Goes to	Wire colour
VCC	+ rail	Red
GND	– rail	Black
SCL	free hole in the D22 column	Yellow
SDA	free hole in the D21 column	Blue
XDA, XCL, AD0, INT	nothing	

Why female-to-male wires: later the sensor gets taped to the blade, away from the breadboard. Long wires let it move.

Why D21 and D22: the sensor talks using a system called **I2C**, which needs a data line (**SDA**) and a clock line (**SCL**). On the ESP32, D21 and D22 are the default I2C pins, so the library finds the sensor without extra setup.

Test it:

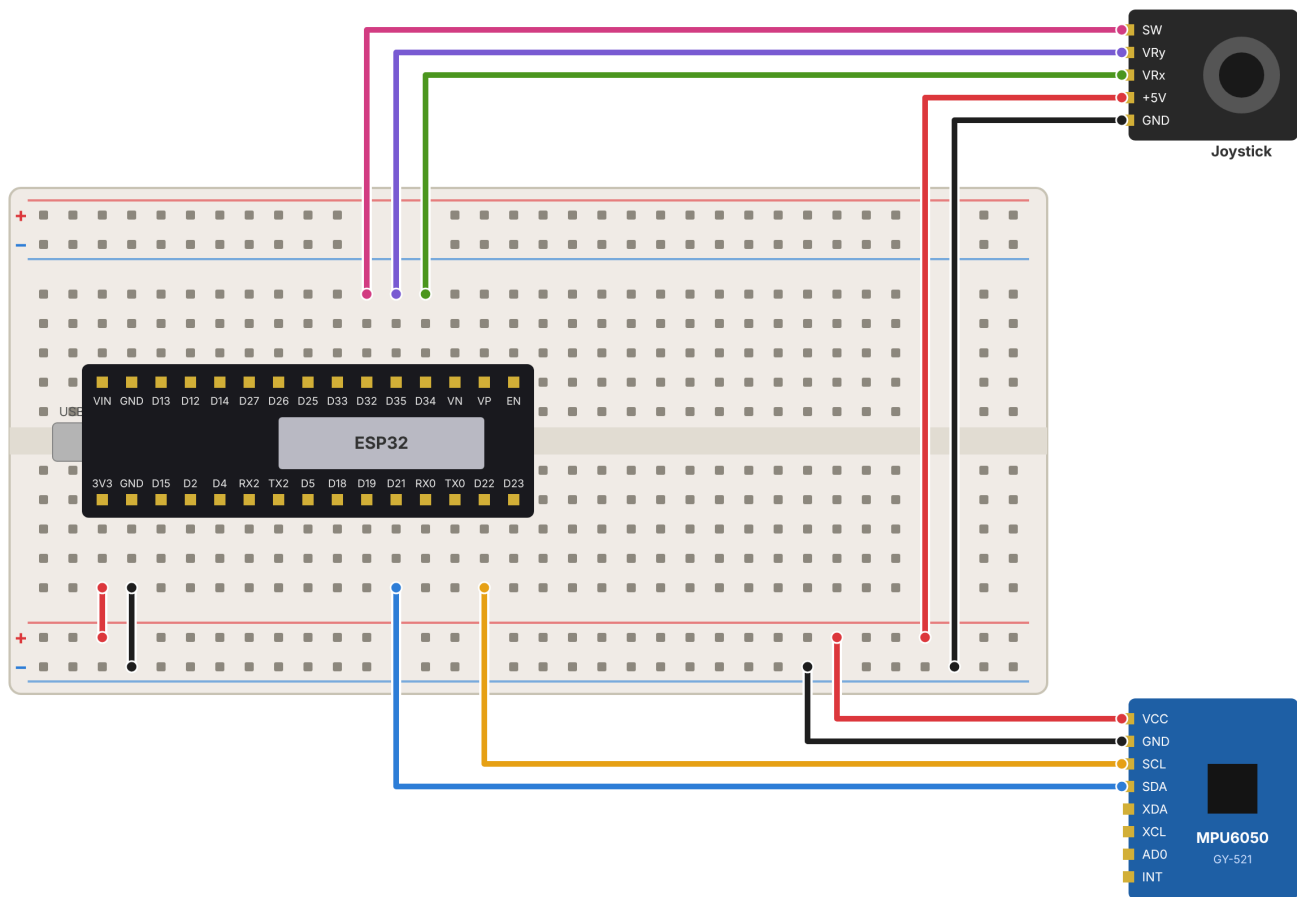
1. Open `sword_test.ino` in the Arduino IDE and click **Upload** (the → arrow). If it gets stuck on *Connecting...*, hold the **BOOT** button on the ESP32 until the upload starts.
2. Open Tools → **Serial Plotter** and set the speed to **115200**.
3. Hold the sensor still. The *total* line should sit near **9.8**. That is gravity.
4. Swing it. *total* should spike to 20 or more.

Why test now: if something is wrong, you only have 4 wires to check instead of 13.

Says "MPU6050 not found"? SDA and SCL are swapped, a wire isn't fully pushed in, or the sensor's VCC/GND aren't on the rails.

Step 5: Wire and test the joystick

Five more female-to-male wires connect the joystick. After this, all the wiring is done.



Joystick pin	Goes to	Wire colour
GND	– rail	Black
+5V	+ rail (the 3.3V one)	Red
VRx	free hole in the D34 column	Green
VRy	free hole in the D35 column	Purple
SW	free hole in the D32 column	Pink

Why +5V goes to 3.3V: the label just means it *can* take 5V. At 3.3V the joystick sends signals the ESP32 can read safely.

What each pin does:

- **VRx** and **VRy** send a number from 0 to 4095 for how far the stick is pushed left/right and up/down. Middle is about 2000.
- **SW** is the click when you press the stick down.

Why D34, D35 and D32: they can read in-between values (analog), and they keep working while Wi-Fi is on. Some other ESP32 pins stop reading analog values when Wi-Fi turns on.

Test it:

1. Open `sword_wireless.ino`. Don't change anything yet, and click **Upload**.
2. Open Tools → **Serial Monitor** at **115200**. **Don't touch the joystick for the first second**. It is measuring where the middle is.
3. You will see lines like `STATE 0 0 0 0` ten times a second. Push the joystick and the first two numbers change to 1 or -1. Press it down and the last number becomes 1.
4. Swing the sensor and a `SWING 1` line appears.

Ignore the *Wi-Fi not connected* message for now. That gets set up in Step 7.

Step 6: Play Minecraft over the USB cable

Get it working with the cable first. If something breaks later with Wi-Fi, you'll know the problem is Wi-Fi and not the sword.

1. **Close the Serial Monitor and Serial Plotter.** Only one program can listen to the ESP32 at a time.
2. **Set up Minecraft:** open a world, put a **sword in your main hand** and a **shield in your offhand** (press F to swap).
 - *Why the shield:* modern Minecraft only blocks with a shield. Holding right click raises the offhand shield, so in your hands it feels like you're guarding with the sword.
3. **Start the bridge.** In a terminal, go to the folder with `sword_bridge.py` and run it with your port name from the Arduino IDE:
 - Windows: `python sword_bridge.py COM3`
 - Mac: `python3 sword_bridge.py /dev/cu.usbserial-0001`
4. **Click into the Minecraft window** so it's the active window. Then swing, guard, walk and jump.

Mac only: the first time, macOS asks to give Terminal *Accessibility* permission (System Settings → Privacy & Security → Accessibility). Turn it on, or the clicks won't happen.

Built-in safety: if the bridge hears nothing from the sword for 1 second, it lets go of every key. Your character won't keep running if a wire pops out.

Step 7: Go wireless

The ESP32 has Wi-Fi built in. Put the sword and the laptop on the same phone hotspot, power the sword from your 9V battery, and the cable is gone.

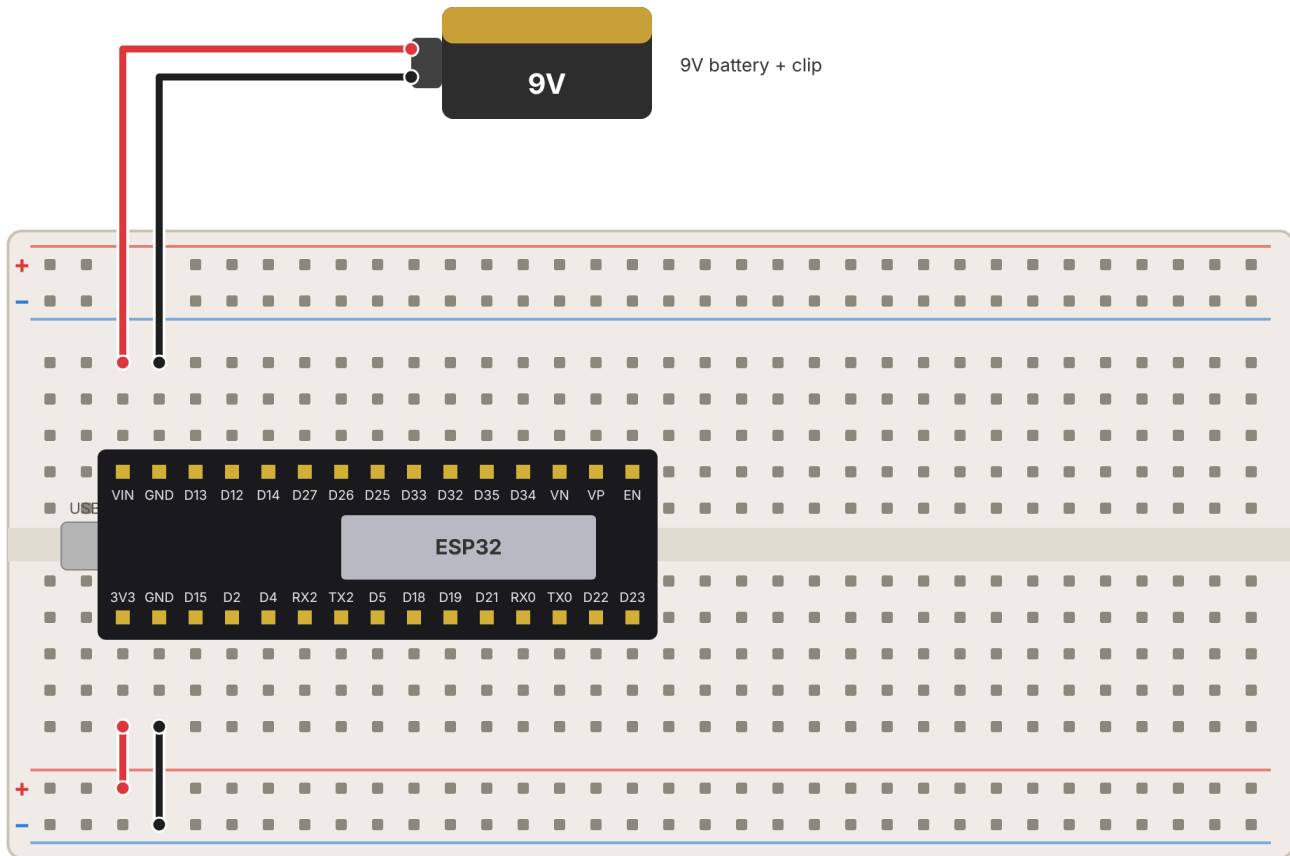
Why a phone hotspot instead of the hackathon Wi-Fi: big event networks often block devices from talking to each other and need a login page the ESP32 can't open. Your own hotspot avoids both problems.

1. **Turn on your phone's hotspot.** On iPhone, turn on *Maximize Compatibility* (it switches to 2.4 GHz). The ESP32 can't see 5 GHz Wi-Fi.
2. **Put the hotspot name and password in the code.** At the top of `sword_wireless.ino`, change `PUT_HOTSPOT_NAME_HERE` and `PUT_HOTSPOT_PASSWORD_HERE`. Keep the quote marks. Upload again.
3. **Check the blue LED on the ESP32.** Blinking means connecting. Solid means connected. Off means it couldn't connect, so check the name and password (capitals matter).
4. **Connect the laptop to the same hotspot.**
5. **Run the bridge in Wi-Fi mode:** `python sword_bridge.py wifi`. When the sword is heard, it prints *Sword found!*
 - Windows may ask whether Python can use the network. Click **Allow**, for both private and public networks.
6. **Unplug the USB cable** from the laptop and **connect the 9V battery as shown below.**

Why it works without typing any IP addresses: the sword shouts each message to every device on the hotspot (a *broadcast*), and only the bridge is listening on port 4210.

Laptop never says "Sword found"? The bridge prints *Laptop IP* when it starts. Put that number in `LAPTOP_IP` in the code, for example `"172.20.10.3"`, and upload again.

Powering it with your 9V battery



You need a **9V battery clip with two bare wire ends** (red and black). The kit's battery cable ends in a round plug, which won't fit in the breadboard. Clips cost a few dollars, and hackathon hardware tables usually have them.

1. **Unplug the USB cable first.**
2. **Red wire** into a free hole in the **VIN** column (top row, the pin closest to the USB port).
3. **Black wire** into a free hole in the **GND** column right next to VIN.
4. Snap the clip onto the battery. The blue LED should come on.

Why VIN: VIN goes through a small part on the ESP32 called a **regulator**, which turns 9V down into the 3.3V the board needs. The 3V3 pin and the + rail come *after* the regulator, so 9V there would destroy the ESP32 and the sensor instantly.

Rules:

- The red battery wire **never** touches the + rail or the 3V3 pin.
- Unplug the battery whenever the USB cable is plugged in. Two power sources at once can damage the board or your laptop's USB port.

- The small black regulator near the USB port gets **warm**. That's normal. If it's too hot to touch, unplug and use a phone power bank in the USB-C port instead.
- A 9V battery is small, so with Wi-Fi on expect roughly an hour or two of play (a rough estimate). Unplug it between tests and bring a spare for judging.

Step 8: Build it into the sword

Mount the parts where they work best, and only after everything works on the table.

Part	Where on the sword	Why there
Motion sensor	Flat on the blade, about a third of the way up from the hilt	The blade moves faster than the handle, so swings are easier to detect
Joystick	On the handle, where your thumb rests	You can walk without letting go
Breadboard + ESP32	Behind the crossguard or on the back of the handle	Short wires to both parts, close to your hand so it doesn't feel tip-heavy
9V battery	Taped along the handle or the pommel	It is the heaviest part, and near your hand it balances the sword

Tips:

- Cut the sword from foam board or two layers of cardboard and paint it in pixel blocks. A sandwich of two layers can hide the wires inside.
- Hot glue or zip-tie the motion sensor so it **can't wobble**. A loose sensor makes fake swings.
- Tape the wires where they leave each part so a hard swing can't yank them out of the breadboard.
- After mounting, redo the guard calibration (Troubleshooting below), because the sensor is now facing a new direction.
- Leave the ESP32's USB port reachable so you can still upload code.

Troubleshooting and tuning

Every setting you might need to change is in the *TUNING* section at the top of `sword_wireless.ino`. Change one number, upload, test.

Set up the block pose (do this once the sensor is on the sword):

1. Upload `sword_test.ino` and open the Serial Plotter.
2. Hold the sword in your guard pose, sideways and still.
3. Look at ax, ay and az. One of them reads about **9.8** or **-9.8**.
4. In `sword_wireless.ino`, set `GUARD_AXIS` to that letter (`'x'` , `'y'` or `'z'`), then upload it again.

Problem	Fix
Swings don't register	Lower <code>SWING_THRESHOLD</code> (try 18)
Attacks when you're just moving	Raise <code>SWING_THRESHOLD</code> (try 26)
One swing hits twice	Raise <code>SWING_COOLDOWN</code> (try 500)
Blocking never turns on	Redo the block pose setup above
Blocks while walking around	Raise <code>GUARD_HOLD_MS</code> (try 400)
Pushing forward walks backward	Set <code>FLIP_FB</code> to <code>true</code>
Left and right are swapped	Set <code>FLIP_LR</code> to <code>true</code>
Walks by itself	Restart the ESP32 without touching the joystick, or raise <code>JOY_DEADZONE</code>
Upload stuck on <i>Connecting...</i>	Hold the BOOT button until it starts
Bridge says port is busy	Close the Serial Monitor and Plotter
Nothing happens in Minecraft	Click into the Minecraft window; on Mac, check Accessibility permission
LED blinks fast and never stops	The sensor isn't found. Check its four wires

Demo day tips

A working, reliable demo beats an extra feature every time.

- ☐ Build a small arena world ahead of time, with zombies and skeletons from spawn eggs, so the demo starts instantly.
- ☐ Let judges swing the sword themselves. Holding it is what makes it memorable.
- ☐ Bring a spare 9V battery and keep the USB cable in your pocket as a backup. Step 6 works if Wi-Fi fails.
- ☐ Record a 30-second video the moment everything works, in case the hardware acts up at judging.
- ☐ Practise a 1-minute pitch: what it is, a live swing, and why it matters (for example, making exercise or physio into a game).